# NeuroMIMO

## Deep Learning-Assisted Channel Estimation for MIMO-OFDM Systems

**Final Project Documentation & Research Summary**

An end-to-end full-stack physical layer platform incorporating synthetic Rayleigh multipath channel generation, LS/LMMSE/Residual DNN estimation, FastAPI computational backends, and React interactive telemetry.

October 23, 2026

# Contents

System Verification & Status . . . . . 2
Project Directory Structure . . . . . 3
Execution & Setup Commands . . . . . 4
REST API Documentation . . . . . 5
Empirical Research Findings . . . . . 6
Final-Year Project Report Outline . . . . . 10
Viva Voce Examination Preparation Guide . . . . . 11

## System Verification & Status

The platform is verified by its automated test suites and production build:

- **Backend Test Suite:** 67 of 67 unit/integration tests passing (`python -m pytest backend/tests -q`). The training smoke test writes to a temporary checkpoint, so running the suite never overwrites the trained model.
- **Frontend Bundle:** Production build passing with 0 TypeScript errors (`npm run build`); `npm run lint` (oxlint) reports warnings only.
- **Live Services:** FastAPI service (`http://127.0.0.1:8000`) and Vite application (`http://localhost:5173`) with `/api` proxy routing.
- **Trained Model:** Residual MLP checkpoint `backend/models/neuromimo_dnn.pt` with 330,112 trainable parameters.

# Project Directory Structure

| | |
|---|---|
| NeuroMIMO/ | |
| ├─ .env.example | Environment configuration template |
| ├─ README.md | Project overview and quick start |
| ├─ requirements.txt | Python dependencies (mirrors backend/) |
| ├─ WMC.pdf | Earlier project documentation |
| ├─ docs/ | This document and its generator script |
| ├─ backend/ | |
| │ ├─ data/experiments.db | SQLite experiment audit (created at runtime) |
| │ ├─ models/neuromimo_dnn.pt | Trained PyTorch Residual DNN checkpoint |
| │ ├─ neuromimo/ | |
| │ │ ├─ config.py | Simulation & model configuration dataclasses/enums |
| │ │ ├─ api/main.py | FastAPI application, routes, background training |
| │ │ ├─ api/schemas.py | Pydantic request/response models |
| │ │ ├─ estimation/base.py | Abstract channel estimator interface |
| │ │ ├─ estimation/ls.py | Least Squares + DFT sinc interpolation |
| │ │ ├─ estimation/mmse.py | Linear MMSE (Wiener) estimator |
| │ │ ├─ estimation/dnn.py | ChannelEstimationNet & DNNEstimator |
| │ │ ├─ estimation/trainer.py | Dataset generation, splitting, training loop |
| │ │ ├─ estimation/train_cli.py | Standalone training command |
| │ │ ├─ evaluation/metrics.py | MSE, NMSE, BER, SER, Wilson / rule-of-three CIs |
| │ │ ├─ evaluation/benchmarks.py | Multi-estimator comparison engine |
| │ │ ├─ evaluation/experiments.py | ExperimentRunner: 5 experiments + figures |
| │ │ ├─ evaluation/run_experiments_cli.py | Reproducible experiment command |
| │ │ ├─ storage/database.py | SQLite persistence for runs and training |
| │ │ ├─ wireless/channel.py | AWGN & exponential-PDP Rayleigh multipath |
| │ │ ├─ wireless/modulation.py | Gray-coded QPSK & 16-QAM (de)mapping |
| │ │ ├─ wireless/ofdm.py | 64-FFT, CP-16, orthogonal comb pilots |
| │ │ ├─ wireless/equalization.py | Zero-Forcing & MMSE MIMO equalizers |
| │ │ └─ wireless/simulation.py | End-to-end MIMOSimulationPipeline |
| │ ├─ results/ | 300 DPI figures, CSV & JSON datasets |
| │ └─ tests/ | Pytest suite (67 tests across 6 files) |
| └─ frontend/ | React 19 + TypeScript + Vite + Tailwind |
| ├─ vite.config.ts | Dev server with /api proxy to port 8000 |
| ├─ public/results/ | Static copies of publication figures |
| └─ src/ | |
| ├─ api/client.ts | Typed REST API client |
| ├─ types/index.ts | Shared TypeScript interfaces |
| ├─ components/common/ | StatCard, Badge, Modal, LoadingSpinner, ErrorBanner |
| ├─ components/layout/ | Navbar (status indicator), Sidebar (drawer) |
| └─ pages/ | Overview, Simulator, ChannelEstimation, ModelTraining, PerformanceComparison, SignalVisualization, ExperimentHistory, About |

# Execution & Setup Commands

## Backend Service

```
# Create and activate a virtual environment (Windows: .venv\Scripts\activate)
python -m venv .venv
source .venv/bin/activate

# Install Python dependencies
pip install -r backend/requirements.txt

# Run the complete test suite
python -m pytest backend/tests -q

# Launch the FastAPI REST service (docs at http://127.0.0.1:8000/docs)
uvicorn neuromimo.api.main:app --app-dir backend --port 8000 --reload
```

## Frontend Service

```
cd frontend

# Install Node packages exactly as locked
npm ci

# Build client production bundle (strict TypeScript check)
npm run build

# Start Vite development server (http://localhost:5173)
npm run dev
```

## Training & Experiment Commands

```
cd backend

# Train the Residual DNN and save backend/models/neuromimo_dnn.pt
python -m neuromimo.estimation.train_cli --samples 500 --epochs 15 --lr 0.003

# Reproduce every experiment, figure and dataset in backend/results/
python -m neuromimo.evaluation.run_experiments_cli --trials 15

# Regenerate this PDF (from the repository root)
python docs/build_documentation.py
```

**CORS:** the API accepts browser requests only from the origins listed in `NEUROMIMO_CORS_ORIGINS` (default: the local Vite dev server).

# REST API Documentation

| Method | Endpoint | Description | Status codes |
|---|---|---|---|
| GET | `/api/health` | Health, device & library versions | 200 |
| GET | `/api/config` | Supported limits & defaults | 200 |
| POST | `/api/simulation/run` | Single-shot MIMO-OFDM simulation | 200 / 400 / 422 |
| POST | `/api/estimation/compare` | Run several estimators on one channel | 200 / 400 / 422 |
| POST | `/api/model/train` | Synchronous or background DNN training | 200 / 400 / 409 / 500 |
| GET | `/api/model/status` | Checkpoint existence & parameters | 200 |
| GET | `/api/model/train/status` | Live epoch-by-epoch loss tracking | 200 |
| POST | `/api/model/predict` | Direct model inference | 200 / 400 / 404 |
| GET | `/api/experiments` | List saved database runs | 200 |
| GET | `/api/experiments/benchmark/snr-sweep` | Pre-computed SNR sweep dataset | 200 / 404 |
| GET | `/api/experiments/{id}` | Detailed run record | 200 / 404 |
| GET | `/api/experiments/{id}/plots` | List publication plots | 200 |
| GET | `/api/experiments/{id}/plots/{name}` | Serve a 300 DPI PNG figure | 200 / 404 |

A 409 is returned when a training job is already running. 422 is FastAPI's automatic validation error for malformed request bodies. Interactive OpenAPI documentation is served at `/docs`.

# Empirical Research Findings

Evaluations use a 2×2 MIMO-OFDM link with QPSK over a 4-tap Rayleigh channel (exponential PDP, decay 1.5), pilot spacing 8, and 15 Monte Carlo trials per SNR step (47,040 payload bits per SNR point). All values below are read directly from the JSON datasets in `backend/results/`.

| Metric / scenario | Least Squares | Linear MMSE | Residual DNN | Genie (known CSI) |
|---|---|---|---|---|
| NMSE @ 0 dB | −8.38 dB | −4.01 dB | −8.36 dB | −∞ (exact) |
| NMSE @ 15 dB | −23.09 dB | −20.50 dB | −20.88 dB | −∞ (exact) |
| NMSE @ 30 dB | −38.64 dB | −38.41 dB | −24.76 dB | −∞ (exact) |
| BER @ 0 dB | 0.2310 (10,866 err) | 0.2459 (11,567 err) | 0.2318 (10,904 err) | 0.2064 (9,710 err) |
| BER @ 15 dB | 0.0307 (1,443 err) | 0.0328 (1,541 err) | 0.0333 (1,567 err) | 0.0259 (1,219 err) |
| BER @ 25 dB | 0.0067 (314 err) | 0.0069 (325 err) | 0.0115 (540 err) | 0.0054 (252 err) |
| BER @ 30 dB | 0.0018 (85 err) | 0.0018 (86 err) | 0.0066 (311 err) | 0.0017 (82 err) |
| Inference latency (mean) | 0.12 ms | 0.35 ms | 0.49 ms | n/a |
| Throughput | 8,635 est/s | 2,879 est/s | 2,053 est/s | n/a |

Table 1: Estimator accuracy, link reliability and latency (Rayleigh, QPSK). Latency: 100 iterations after 10 warm-up runs on CPU.

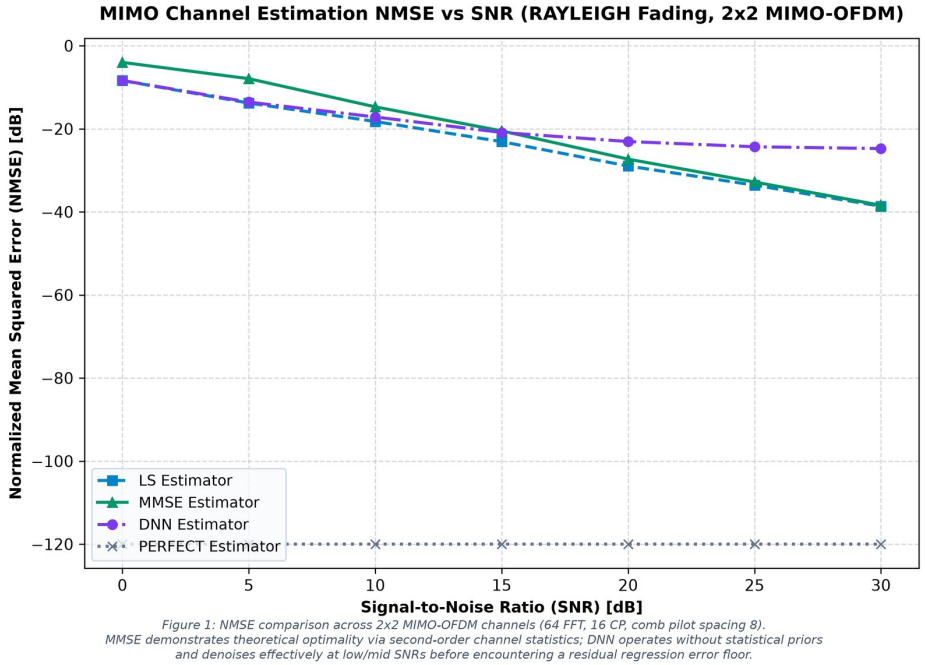



Figure 1: Channel estimation NMSE versus SNR.

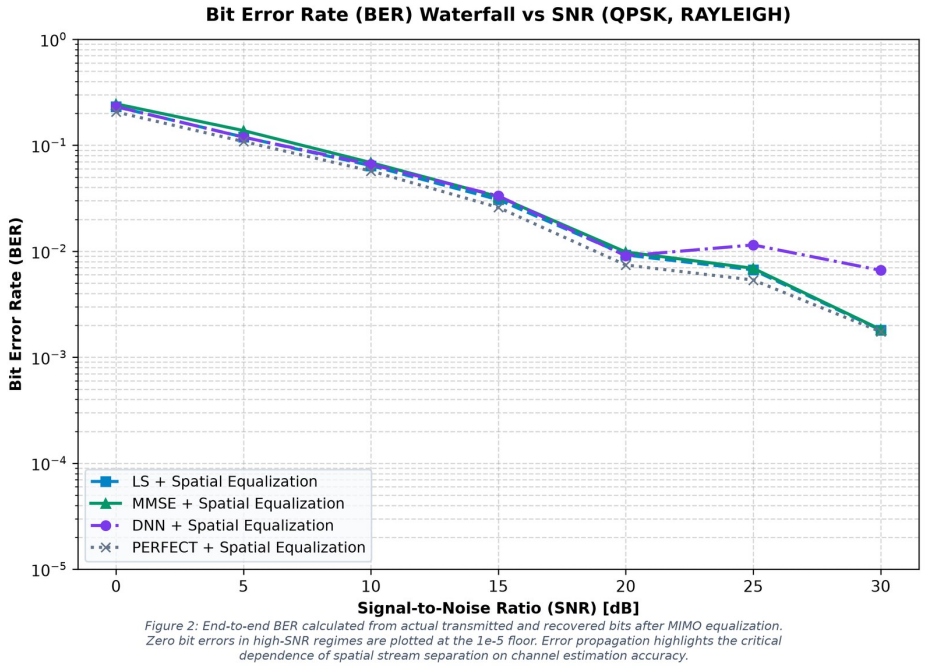



Figure 2: Bit error rate versus SNR with 95% confidence intervals.

## Engineering Analysis

- **Low SNR (0 dB):** the Residual DNN (−8.36 dB) matches LS (−8.38 dB) and is about 4.3 dB better than this LMMSE implementation (−4.01 dB), without needing a channel covariance matrix or noise power estimate at inference time.
- **Mid SNR (15 dB):** LS leads (−23.09 dB), followed by the DNN (−20.88 dB) and LMMSE (−20.50 dB). BER differences are small: 0.0307 / 0.0328 / 0.0333.
- **High-SNR error floor (> 20 dB):** the DNN saturates near −24.76 dB at 30 dB, whereas LS with DFT sinc interpolation keeps improving to −38.64 dB. This comes from training one lightweight 4-layer MLP (330k parameters) across the whole 0–30 dB range; at 30 dB the DNN makes 311 bit errors versus 85 for LS.
- **Latency:** LS is the fastest estimator; LMMSE costs about 3× and the DNN about 4× the LS time, all well under 1 ms per frame on CPU.
- **Statistical honesty:** every BER point carries a 95% Wilson score interval, and zero-error points are reported as an upper bound using the Poisson rule of three ($3/N_{bits}$), never as a misleading 0.000.

## Propagation Environment Sensitivity

| AWGN channel | Least Squares | Linear MMSE | Residual DNN | Genie (known CSI) |
|---|---|---|---|---|
| NMSE @ 0 dB | −8.38 dB | −8.93 dB | −8.12 dB | −∞ (exact) |
| NMSE @ 15 dB | −23.08 dB | −23.23 dB | −17.90 dB | −∞ (exact) |
| NMSE @ 30 dB | −38.58 dB | −38.58 dB | −19.69 dB | −∞ (exact) |
| BER @ 0 dB | 0.1680 (7,903 err) | 0.1585 (7,455 err) | 0.1700 (7,999 err) | 0.1504 (7,073 err) |
| BER @ 15 dB | < 6.4e-05 (0 err) | < 6.4e-05 (0 err) | < 6.4e-05 (0 err) | < 6.4e-05 (0 err) |

Table 2: Estimator behaviour on a flat AWGN channel. The DNN, trained on Rayleigh multipath, shows an even earlier NMSE floor on this out-of-distribution channel, although BER is unaffected at mid/high SNR.

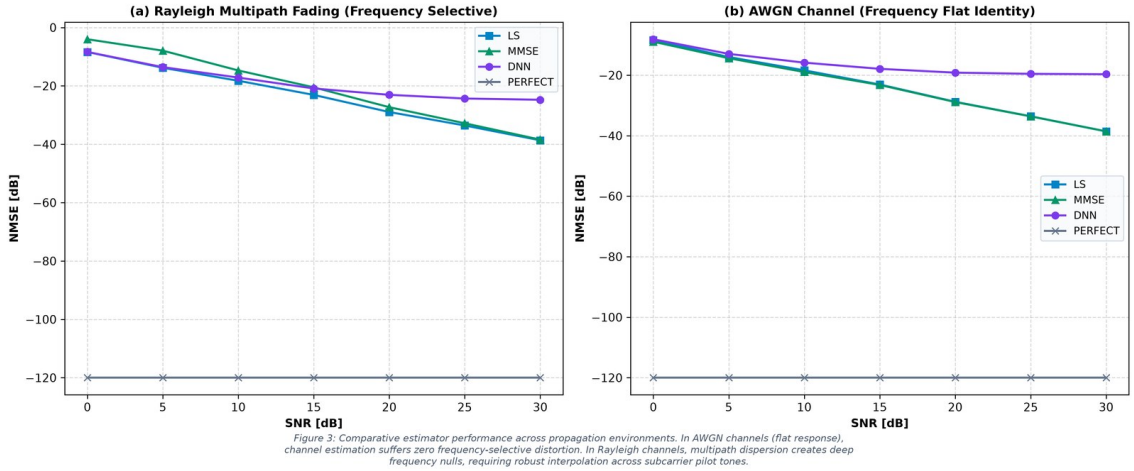



Figure 3: AWGN versus Rayleigh multipath estimator comparison.

## Generalization to Unseen Channels

| Condition (15 dB) | Seen in training | LS | LMMSE | Residual DNN | Genie |
|---|---|---|---|---|---|
| PDP Decay 0.8 | no | −24.01 dB / 0.0261 | −20.22 dB / 0.0283 | −21.62 dB / 0.0304 | exact / 0.0237 |
| PDP Decay 1.5 | yes | −24.00 dB / 0.0289 | −21.16 dB / 0.0302 | −21.59 dB / 0.0298 | exact / 0.0257 |
| PDP Decay 2.5 | no | −23.99 dB / 0.0294 | −21.58 dB / 0.0298 | −21.54 dB / 0.0314 | exact / 0.0264 |
| 6 Taps Dispersion | no | −9.38 dB / 0.1186 | −9.69 dB / 0.1172 | −9.39 dB / 0.1213 | exact / 0.0314 |

Table 3: NMSE / BER under power-delay-profile mismatch (training decay 1.5). All estimators degrade sharply with 6 taps because the comb pilot spacing of 8 can no longer resolve the longer channel impulse response.

### Pilot Density Trade-off

| Pilot spacing | Pilots / Tx | LS | LMMSE | Residual DNN |
|---|---|---|---|---|
| 4 | 8 | −23.92 dB | −25.13 dB | −21.01 dB |
| 8 | 4 | −23.09 dB | −20.50 dB | −20.88 dB |
| 16 | 2 | −3.95 dB | −4.65 dB | −3.95 dB |

Table 4: NMSE at 15 dB versus comb pilot spacing. Spacing 16 leaves only 2 pilots per transmit antenna, below the 4 needed to sample a 4-tap channel, and every estimator collapses.

## Known Limitations & Roadmap

- **High-SNR floor:** SNR-conditioned inputs, a deeper network or per-SNR-band fine-tuning would remove the DNN saturation above 20 dB.
- **2D time-frequency correlation:** the DNN processes each OFDM frame independently. A 2D complex CNN or Vision Transformer would exploit joint delay-Doppler correlation.
- **High-mobility Doppler:** the channel is block-fading per frame. Jakes-spectrum time variation would allow evaluating inter-carrier interference compensation.
- **Massive MIMO scaling:** the platform is configured for 2×2 spatial multiplexing; larger arrays with hybrid beamforming are a natural extension.
- **Configuration:** only `NEUROMIMO_CORS_ORIGINS` is read from the environment today; the remaining `.env.example` values document built-in defaults.

# Final-Year Project Report Outline

## Chapter 1: Introduction

1.1 Wireless Evolution: 5G NR, Wi-Fi 6/7, and the Path to 6G

1.2 Problem Statement: Channel Estimation in Frequency-Selective Fading

1.3 Project Motivation & Objectives of NeuroMIMO

1.4 Scope, Technical Contributions & Report Organization

## Chapter 2: Theoretical Foundations & Literature Review

2.1 MIMO Spatial Multiplexing Principles

2.2 OFDM Baseband Transmission, Cyclic Prefix & Subcarrier Orthogonality

2.3 Classical Estimation: Least Squares & Wiener-Hopf MMSE

2.4 Deep Learning in Physical Layer Communications: Opportunities & Pitfalls

## Chapter 3: System Design & Implementation

3.1 Vectorized MIMO-OFDM Transceiver Architecture

3.2 Comb Pilot Pattern Design & Alias-Free DFT Sinc Interpolation

3.3 PyTorch Residual Deep Neural Network Architecture

3.4 Data Leakage Prevention & Supervised Training Formulation

3.5 Full-Stack Software Engineering (FastAPI, React 19, SQLite)

## Chapter 4: Experimental Methodology & Results

4.1 Monte Carlo Simulation Setup & Statistical Rigor (Wilson Intervals & Poisson Bounds)

4.2 Channel Estimation Accuracy (NMSE vs SNR)

4.3 End-to-End Demodulation Performance (BER vs SNR with MMSE Equalization)

4.4 Propagation Environment Sensitivity (AWGN vs Rayleigh Multipath Fading)

4.5 Generalization to Unseen Channels & PDP Perturbations

4.6 Pilot Density Trade-off

4.7 Real-Time Inference Latency & Computational Throughput Profiling

## Chapter 5: Conclusion & Future Work

5.1 Summary of Findings

5.2 Critical Engineering Trade-Offs (Residual Floor vs Computational Simplicity)

5.3 Future Directions: Complex-Valued CNNs, Fast Doppler Tracking, Massive MIMO

# Viva Voce Examination Preparation Guide

**Q1: Why does Least Squares (LS) suffer from noise enhancement at low SNR?**

LS estimation performs element-wise division at each pilot tone:

$$\hat{H}_{LS}[p] = Y[p] / X[p] = H[p] + W[p] / X[p]$$

Because it uses no statistical prior on the channel covariance or noise variance, the noise power in the estimate equals $\sigma_n^2 / |X[p]|^2$. At low SNR this noise passes directly into the channel estimate and degrades the downstream MIMO equalizer.

**Q2: What is the main practical drawback of Linear MMSE?**

LMMSE needs the channel autocorrelation matrix $\mathbf{R}_{HH}$ and the noise variance $\sigma_n^2$, both of which must be tracked in time-varying environments. It also needs the matrix inverse $(\mathbf{R}_{HpHp} + \sigma_n^2\mathbf{I})^{-1}$, with $O(N_p^3)$ complexity, which adds latency. In this project its mean latency is about 3× that of LS.

**Q3: Why use a residual network instead of predicting the full channel?**

Predicting the channel directly forces the model to learn the transfer function and the noise removal at the same time. With a residual connection

$$\hat{H}_{DNN} = \hat{H}_{LS} + f_\theta(\hat{H}_{LS})$$

the network only learns the correction $\Delta H = H_{true} - \hat{H}_{LS}$. Starting from small weights, it begins at LS accuracy and refines it, which speeds up convergence and avoids vanishing gradients.

**Q4: How did you prevent data leakage between training and testing?**

The dataset is split by independent channel-realization seeds before features are extracted, so test channels are never seen in training. Input features come only from noisy received pilots and LS interpolation; the true channel is used only as the regression target.

**Q5: Why is the Poisson rule of three used in the BER evaluation?**

With a finite number of bits (47,040 per SNR point), observing zero errors does not mean the true error probability is zero. The rule of three gives a 95% upper bound:

$$p_{95\%} \leq 3 / N_{bits} = 3 / 47{,}040 \approx 6.38\text{e-}05$$

Non-zero points use the Wilson score interval, and a point is flagged as statistically conclusive only when at least 10 errors were observed.

**Q6: Why does the DNN lose to LS above 20 dB?**

One small network is trained across 0–30 dB, so it learns an average denoising behaviour. At high SNR the LS estimate is already very accurate, and the network's own approximation error becomes the dominant error, producing a floor near −24.76 dB. SNR-aware inputs or per-band models are the fix.